Document number:	P2487R1
Date:	2023-06-11
Audience:	SG21
Reply-to:	*Andrzej Krzemieński <akrzemi1 at gmail dot com>*

Is attribute-like syntax adequate for contract annotations?

This paper is an attempt to structure the discussion on the choice of syntax for contract annotations.

[P0542] and [P2388R4] propose a syntax that appears similar to attributes:

```
int f(int i)
  [[pre: i >= 0]]
  [[post r: r >= 0]];
```

[P2461] proposes, similarly to [N1962], an alternate syntax, nothing like attributes:

```
int f(int i)
  pre{i >= 0}
  post(r){r >= 0};
```

And we could invent more syntax alternatives, for instance:

```
int f(int i)
  pre(i >= 0)
  post(r: r >= 0);
```

Rather than comparing different syntax choices, we try to answer the question: is attribute-like syntax suitable for contract annotations. It is clear that contract annotations using attribute-like syntax proposed in [P2388R4] are not attributes according to the grammar productions of C++. Instead, the question we try to answer is: do we want the programmers to *think* of them as attributes.

Revision history {rev}

R0 → R1 {rev.r01}

1. Updated the discussion on ignorability, based on [P2552R2] and the revised C++ Standard ([N4944]).
2. Added discussion of expressions in attributes.
3. Added discussion on semantic ignorability criterion, presented in [P2552R2].

"Ignorable" {ign}

Interestingly, one argument brought up in favor of the attribute-like syntax is that contract annotations are "ignorable", whereas one argument against the attribute-like syntax say that contract annotations are not "ignorable". This indicates that we do not have a shared understanding of what "ignorable" means.

We can get the best approximation of the term from the C and C++ standards. The C Standard ([N2596]) has the following.

6.7.11 p2:

> Support for any of the standard attributes specified in this document is implementation-defined and optional. For an attribute token (including an attribute prefixed token) not specified in this document, the behavior is implementation-defined. Any attribute token that is not supported by the implementation is ignored.

4 p5:

> A strictly conforming program shall use only those features of the language and library specified in this document. It shall not produce output dependent on any unspecified, undefined, or implementation-defined behavior, and shall not exceed any minimum implementation limit.

6.7.11.1 p3:

> [...] A strictly conforming program using a standard attribute remains strictly conforming in the absence of that attribute. [166)]

Footnote 166:

> Standard attributes specified by this document can be parsed but ignored by an implementation without changing the semantics of a correct program; the same is not true for attributes not specified by this document

The C++ Standard [N4944] has the following to say:

9.12.1 p1:

> Attributes specify additional information for various source constructs such as types, variables, names, blocks, or translation units.

9.12.1 p6:

> For an *attribute-token* [...] not specified in this document, the behavior is implementation-defined; any such *attribute-token* that is not recognized by the implementation is ignored.
>
> [*Note 4*: A program is ill-formed if it contains an `attribute` specified in 9.12 that violates the rules specifying to which entity or statement the attribute can apply or the syntax rules for the attribute's `attribute-argument-clause`, if any. — *end note*]

The definition in C does not make it quite clear if "ignore" means "must parse correctly, but has no semantics", or "does not even have to parse correctly". For instance, is an implementation allowed to accept the following code without emitting any diagnostic?

```
void f(auto [[noreturn(int)]] i);
```

The C++ definition, on the other hand, makes it clear that "ignore" means "must parse correctly, but has no semantics".

Let's introduce two definitions to tell apart these two interpretations of "ignorability":

Syntactic ignorability
: This is the WG14 model. An implementation, while doing the parsing of the source code can ignore any balanced sequence of tokens between `[[` and `]]`.

Semantic ignorability
: An implementation is required for every standard attribute to verify at compile-time if all the constraints corresponding to the attribute are satisfied, such as what it appertains to, how many times it occurs in the attribute-list, does it have the correct number and types of parameters. It is also required to ODR-use entities

in the expression if it is part of attribute's argument list. Other than that an implementation can translate the program as if the attribute was absent. This is discussed in more detail in [P2552R2].

While the syntactic ignorability may disincetivize the implementations from detecting bugs in the attribute usage, it also provides a better backwards compatibility. For instance, the following code is correct in C++17:

```
[[nodiscard("get ownership")]] int * allocate();
```

However, if we compiled it with C++14 we would get a compiler error when syntactic conformance of attributes is verified. This is because in C++14 attribute `nodiscard` was recognized but was allowed no argument clause.

The semantic ignorability corresponds to the semantics of contract annotations in translation mode *No_Eval*, as described in [P2388R4]: the syntactic and type-system correctness are verified, the entities in the predicate are ODR-used, but other than that nothing happens.

Parsing expressions {exp}

It was reported in the past that having to parse C++ expressions inside attributes is challenging because attributes were meant to be simple annotations to be parsed at earlier stages of translation, where no type system information is present yet.

The requirement to parse C++ expressions inside `[[_]]` is implementable. The implementation would have to recognize the sequence `[[pre: _]]` as special, and treat it distinctively, as a keyword. However, the requirement to parse expressions breaks the simple model for attributes: they can no longer be parsed at earlier phases of translation: their well-formedness now depends on the type system.

In response to this, we could say that C++ are not attributes, but a different feature that also happens to use the `[[_]]` notation. More, since C++23 the language has now attribute `[[assume(_)]]` which already requires parsing C++ expressions. See [P1774R8].

"Declarative" vs "imperative" {dvi}

There are (at least) two possible models for describing semantics contract annotations. The first is that they are predicates (in the mathematical sense) for describing what constitutes an incorrect program at a given point in program execution. We can call it a "declarative" model. The other model is that contract annotations give a programmer a tool to execute arbitrary code at certain points in program execution, which can include control flows, such as `throw`-expression. We can call it an "imperative" model. An example of such model is proposed in [P1893], and it takes contract support close to `call` and `return` features described in [STROUSTRUP] and [DnE] .

The preference of either of the models also affects the choice of the syntax for contract annotations. In the declarative model, the attribute-like syntax is a well suited candidate. But it is counterintuitive if we adopt the imperative model.

"Reorderable" {reo}

Although it is not specified anywhere formally, the intuitive expectation of the attributes is that while you can reorder two attributes inside a single *attribute-specifier* without changing the meaning, you cannot safely reorder two *attribute-specifier*s:

```
[[a, b]] int f(); // same as [[b, a]] int f();

[[c]] [[d]] int g(); // different than [[d]] [[c]] int g();
```

This is compatible with contract annotations. One cannot reorder the following two contract annotations, as the short-circuiting property would be compromised:

```
int f(int * p)
  [[pre: p != nullptr]]
  [[pre: *p > 0]];
```

Ordering {ord}

Today, it is difficult to remember in which order declarators like `noexcept`, `const ->` and attributes should appear in function declarations after the parentheses:

```
int f(int) const noexcept [[gnu::fastcall]]; // correct or ill-formed?
```

Adding a new declarator to the list, looking similar to attributes, will introduce a new: sort of dilemma: how these contact annotations should be ordered relative to attributes appertaining to function type:

```
int f1(int i)               // correct declaration?
  [[pre: i > 0]]
  [[using gnu: fastcall]]
  [[post r: r > 0]];

int f2(int i)               // correct declaration?
  [[using gnu: fastcall]]
  [[pre: i > 0]]
  [[post r: r > 0]];

int f3(int i)               // correct declaration?
  [[pre: i > 0]]
  [[post r: r > 0]]
  [[using gnu: fastcall]];
```

`[[_]]` as container {cnt}

The present intuition that people might adopt is that two pairs of square brackets (an *attribute-specifier*) is a "container" that can hold zero, one or more comma-separated attributes:

```
[[]] int f(); // ok: zero attributes
[[noreturn]] int g(); // ok: one attribute
[[noreturn, nodiscard]] int h(); // ok: two attributes
```

Contract annotations would not fit into this intuition.

Type and Effect analysis {tea}

Imagine a system of annotations -- `[[writable]]`, `[[readable]]` -- that helps track the lifetime of a heap-allocated objects:

```
int* [[writable]] allocate ();
int* [[readable]] fill(int* [[writable]] p);
void read(int* [[readable]] p);
void deallocate(int* [[writable]] p);
```

Thus, function `deallocate()` requires the pointer to be `writable`. `readable` implies `writable`. Function `allocate()` produces a `writable` value. Function `fill()` upgrades the `writable` property to `readable`. This constitutes an *annotated type system* that can be subject to *Type an Effect analysis*. A tool can try to determine if `deallocate()` or `fill()` is ever called with a pointer that is not `writable`, or if `read()` is ever called with a pointer that is not `readable`.

We could invent a different system of annotations for tracking the ownership of a resource:

```
Res* [[in_session]] acquire ();
void use(Res* [[in_session]] r);
void release(Res* [[in_session, ends_session]] p);
```

This constitutes another Effect system, and a similar analysis could be performed using this system. And we could invent more and more such systems. The attributes are used to extend the type system with the effects system. The effects system is not enforced by the compiler, but is formal enough to enable a certain kind of analysis. The attribute syntax is a natural choice for expressing these effects.

Now, one of the use cases for contract support framework is to provide an automated way for describing the effect systems:

```
axiom writable(int*);
axiom readable(int*);

int* allocate()  [[post r: writable(r)]];
int* fill(int* p) [[pre: writable(p)] [[post r: readable(r)]];
void read(int* p) [[pre: readable(p)]];
void deallocate(int* p) [[pre: writable(p)]];
```

In this example, we use keyword `axiom` as proposed in [P2176R0]. With this capability, one can build a new kind of tool for performing Type and Effect analysis: one that is not tied to a fixed set of annotations, but can be taught to recognize arbitrary effect systems. Given this use case — guiding type an effect static analysis — the attribute-like syntax looks natural.

The desire to support something like this has been reflected in [P1995] as the following use cases:

- api.express.unimplementable,
- dev.tooling.

Appertaining to function type {apr}

One argument against the attribute-like syntax, is that:

1. the position they are at means for attributes that they appertain to function type, and
2. they do not appertain to function type.

In order to validate this claim we would have to have a clear criterion for what qualifies as "appertaining to function type". The C++ or C Standards do not give us an answer. We could reach to non-standard attributes. One case highlighted by Aaron Ballman is a vendor-specific attribute `[[gnu::fastcall]]`:

```
void func(int i) [[gnu::fastcall]];

int main() {
  void (*fp)(int) = func; // error: type mismatch
}
```

See the Compiler Explorer example: https://godbolt.org/z/3YzGb897f. This illustrates that this specific attribute changes the type of the function it appertains to. But is it a general rule? Affecting the Effect system could be a natural generalization of extending the type system.

One practical criterion for appertaining to function type is if we can use the same attribute to declare the function type rather than a function:

```
using AllocFun = void*(size_t s) [[pre: c > 0u]][[post r: r != nullptr]];

AllocFun quickAlloc, smartAlloc; // two functions with contract annotations?
```

Annotations on annotations {met}

While preconditions feel like annotations on functions (they do not affect semantics of "contract-correct" programs"), they themselves require to be annotated with some hints, such as that the predicate in the precondition is more expensive to evaluate than the function guarded by the precondition, or that a precondition annotation is new and is as likely to be incorrect as the code that it is guarding.

Such "annotations" could naturally be expressed as attributes appertaining to preconditions. However, this becomes impossible when preconditions themselves look like attributes: you cannot have an attribute appertain to an attribute. This problem does not exist if a different syntax is used for contract annotations. The following, is an example taken from [P2461]:

```
void store(P* p)
  pre{p != nullptr}
  [[audit]] pre{p->is_square()};
```

If attribute-like syntax is used for contract annotations, we would have to invent a new, unprecedented, syntax for annotating annotations:

```
int get_val()
  [[post audit new val: good(val)]]    // maybe like this
  [[post val: fine(val); audit, new]]; // or maybe like this
```

Detectability in reflection {rfl}

It is possible that we might want in the future to inspect contract annotations through reflection. The question is: should things declared through attributes (or things pretending to be attributes) be detectable through reflection?

The usage of colon {col}

Currently the syntax for *attribute-specifier* uses colon for separating the *attribute-using-prefix* from namespace-less attributes:

```
[[using clang: not_tail_called, optnone]] int f(int i);
```

Re-using colon in post-/pre-condition annotations would overload the meaning of the colon:

```
upsell greatest_negotiable_upsell(list const& l)
  [[post gnu: l.contains(gnu)]]
  [[using gnu: fastcall]];
```

The above example also illustrates that the presence of the colon alone cannot be used to easily tell apart contract annotations and *attribute-specifier*s.

Conclusion {end}

The *semantic ignorability* property of C++ attributes is compatible with contract annotations and their two translation modes. However, contract annotations are not compatible with the *syntactic ignorability* preferred in WG14 for C attributes. Thus, while the attribute-like syntax would be a relatively good fit for C++ it would not necessarily be so for C. Moreover, contract annotations completely disregard the syntactic rules of attributes: appertainment, aggregation, comma-separation. If we adapted the attribute-like syntax, we would get two noticeably different features sharing the notion of optionality reflected in the double square brackets.

Acknowledgments {ack}

Jens Maurer and Aaron Ballman offered substantial feedback on the intuition behind the attribute notation. Tom Honermann and Timur Doumler reviewed the document and offered a substantial feedback.

References {ref}

- [DnE] — Bjarne Stroustrup, "The Design and Evolution of C++", Addison-Wesley, ISBN 0-201-54330-3.
- [STROUSTRUP] — Bjarne Stroustrup, "Wrapping C++ Member Function Calls" (https://www.stroustrup.com/wrapper.pdf).
- [N1962] — Lawrence Crowl, Thorsten Ottosen, "Proposal to add Contract Programming to C++ (revision 4)" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n1962.html).
- [N4944] — Thomas Köppe, "Working Draft, Standard for Programming Language C++" (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4944.pdf).
- [P0465] — Lisa Lippincott, "Procedural function interfaces" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0465r0.pdf).
- [P0542] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html).
- [P1774R8] — Timur Doumler, "Portable assumptions" (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1774r8.pdf).
- [P1893] — Andrew Tomazos, "Proposal of Contract Primitives" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1893r0.pdf).
- [P2176R0] — Andrzej Krzemieński, "A different take on inexpressible conditions" (https://isocpp.org/files/papers/P2176R0.html).
- [P2388R4] — Andrzej Krzemieński, Gašper Ažman, "Minimum Contract Support: either *No_eval* or *Eval_and_abort*" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2388r4.html).
- [P2461] — Gašper Ažman, Caleb Sunstrum, Bronek Kozicki, "Closure-Based Syntax for Contracts" (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2461r0.pdf).
- [P2552R2] — Timur Doumler, "On the ignorability of standard attributes" (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2552r2.pdf).